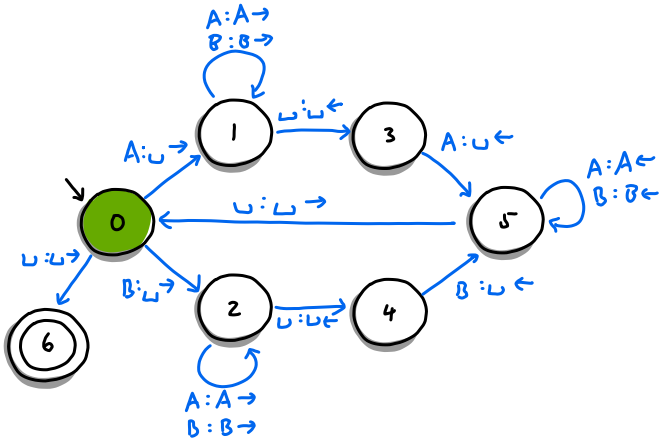
TheoInf 1
Woche 12

Nochmal

Turingmaschine

▸ **Beispiel 2**

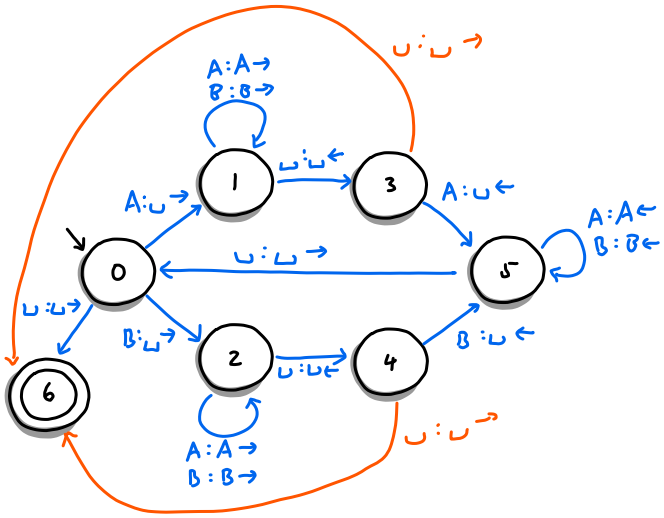
Σ = {A, B}
Q = {0, 1, 2, 3, 4, 5, 6}
s = 0
F = {6}
Γ = {A, B, ␣}





▸ **Frage 1:** Welche Sprache akzeptiert diese Turingmaschine?
L = {w ∈ Σ* | w ist ein Palindrom und w hat eine gerade Länge}

▸ **Frage 2:** Wie muss man die Maschine verändern, damit sie folgende Sprache akzeptiert:
L = {w ∈ Σ* | w ist ein Palindrom}



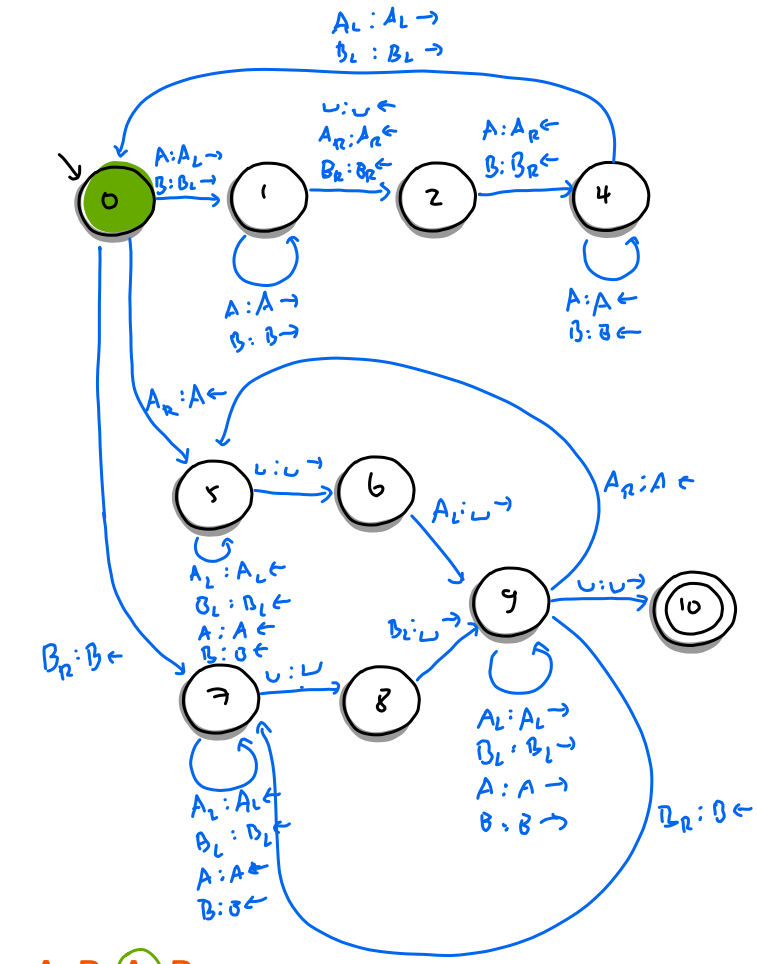
▸ **Beispiel 3:**

$L = \{ww \mid w \in \{A, B\}^*\}$

Idee: Markiere zunächst die Buchstaben im Eingabewort mit "L" oder "R", je nachdem, ob sie in der linken oder in der rechten Hälfte liegen. Vergleiche danach die beiden Hälften.

$\Sigma = \{A, B\}$

$\Gamma = \{A, B, \sqcup, A_L, A_R, B_L, B_R\}$



$\sqcup A_L B_L A_R B_R \sqcup$

⇦ **Hausaufgabe:** Prüfen Sie, ob das so korrekt ist! 😜

Allgemeine Grammatiken

Zur Erinnerung:

▸ **Kontextfreie Grammatiken**

Produktionsregeln haben die Form: $X \to u$
mit $X \in N$ (also ein Nichtterminal)
$u \in (N \cup T)^*$ (ein String aus Terminalen und Nichtterminalen)

▸ **allgemeine Grammatik**

Produktionsregeln haben die Form: $aXb \to aub$
mit $X \in N$ und $a, b, u \in (N \cup T)^*$

- a und b sind der *Kontext*: Nur wenn das X von a und b eingefasst ist, darf X durch u ersetzt werden.
- Dabei können a und b auch ε sein.
- Ersetzt wird aber nur das X!
- Um die Lesbarkeit zu erhöhen, unterstreichen wir X und u.
- Wenn man zusätzlich fordert, dass $u \neq \varepsilon$ ist, wenn a und $b \neq \varepsilon$ sind, dann nennen manche Leute das auch "*kontextsensitive* Grammatik"

▸ **Beispiel:**

$\underline{S} \to \underline{aSXY} \mid \underline{\varepsilon}$
$\underline{Y}X \to \underline{X}\underline{C}X$
$C\underline{X} \to C$
$a\underline{X} \to a\underline{b}$
$b\underline{X} \to b\underline{b}$
$\underline{Y} \to \underline{c}$
$\underline{C} \to \underline{c}$

Beispielablauf:

String	Produktionsregel
S	$\underline{S} \to \underline{aSXY}$
aSXY	$\underline{S} \to \underline{aSXY}$
aaSXYXY	$\underline{S} \to \underline{\varepsilon}$
aaXYXY	$\underline{Y}X \to \underline{X}\underline{C}X$
aaXXCXY	$C\underline{X} \to C$
aaXXCY	$a\underline{X} \to a\underline{b}$
aabXCY	$b\underline{X} \to b\underline{b}$
aabbCY	$\underline{C} \to \underline{c}$
aabbcY	$\underline{Y} \to \underline{c}$
aabbcc	

aaaXXCXXCXXC

Diese Grammatik sollte eigentlich die Sprache $L = \{a^n b^n c^n \mid n \in \mathbb{N}_0\}$ erzeugen, das tut sie aber nicht! (Frage: Wieso?)

Hausaufgabe: Versuchen Sie, eine (allgemeine) Grammatik zu konstruieren, die tatsächlich die Sprache $L = \{a^n b^n c^n \mid n \in \mathbb{N}_0\}$ erzeugt!

► **Wichtig:** Nur Strings, die keine Nichtterminale mehr enthalten, gehören zur Sprache. Wenn es keine anwendbare Regel mehr gibt, mit der man ein Nichtterminal ersetzen kann, dann ist die Produktion gescheitert.

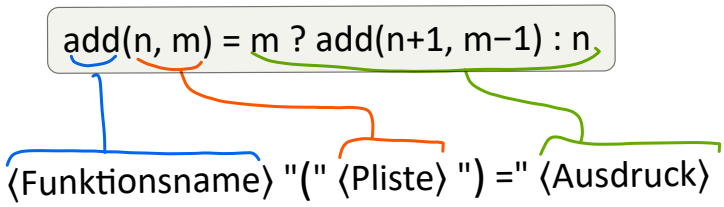
Reterna

Recursive Functions with Ternary Conditional Operator

Reterna ist *rekursiv* ⇒ im Ablauf der wechselseitigen Aufrufe von Funktionen können Zyklen sein.

Loop/While/Goto sind *iterativ* ⇒ sie verwenden Schleifen.

▸ Definition einer Reterna-Funktion



▸ **Wichtig:** Es gibt keine Variablen, sondern nur Parameter

Parameter können nicht verändert werden − aber bei verschiedenen Aufrufen können die Parameter verschiedene Werte haben.

▸ Beispiel:

add(5, 3) = add(6, 2) = add(7, 1) = add(8, 0) = 8

⇦ das ist *endrekursiv*
= jeder Funktionsaufruf startet höchstens einen rekursiven Aufruf, und das ganz am Schluss

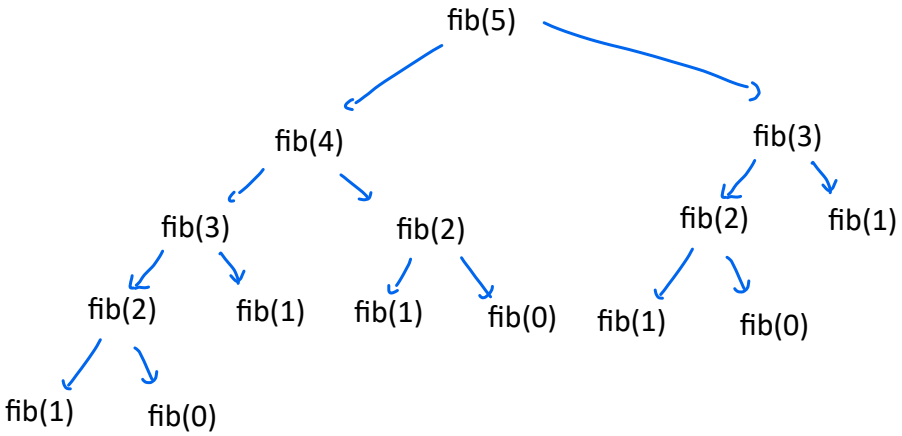
▸ Beispiele für nicht endrekursiv:

`fib(n) = n > 1 ? fib(n−1) + fib(n−2) : 1` ⇦ mehr als ein rekursiver Aufruf

`mul(n, m) = m ? mul(n, m−1) + n : 0` ⇦ nach dem rekursiven Aufruf kommt noch das +

nicht endrekursiv ist schwieriger zu bewältigen, weil man sich bei der Abarbeitung der Funktion verschiedene Dinge merken muss, nämlich:

1. die Parameter aller früheren, noch nicht abgeschlossenen Aufrufe
2. bei mehreren rekursiven Aufrufen: die Stelle, an der man nach Ende des rekursiven Aufrufs weiter machen muss − ebenfals für alle noch nicht abgeschlossenen Aufrufe



Geeigneter Datentyp, um das zu speichern: Stapel ("Stack")

"Stack Overflow" ⇒ Fehler, der passiert, wenn ein Programm zu viele rekursive Aufrufe macht, so dass der Speicherplatz für den Stack nicht mehr reicht.